

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«ВЛАДИВОСТОКСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ» (ФГБОУ ВО
«ВВГУ»)

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И АНАЛИЗА ДАННЫХ КАФЕДРА
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И СИСТЕМ

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ FINAL АЗС
по дисциплине
«Информатика и основы программирования»

Студент
гр. БИН-25-3

С.Д. Мурадуллаева

Ассистент
преподавателя

М.В. Водяницкий

Задание

Техническое задание - Система управления автозаправочной станцией.

Вы работаете в российской компании «НефтеСофт», разрабатывающей программное обеспечение для автоматизации автозаправочных процессов. Вам поручено создать консольный прототип системы управления АЗС, который будет использоваться операторами и техническим персоналом станции.

Система должна учитывать реальные процессы работы по заправке: продажа топлива, контроль запасов, обслуживание цистерн, аварийные ситуации и ведение статистики.

1. Общая идея программы

Программа представляет собой консольную систему управления автозаправочной станцией, которая позволяет:

- обслуживать клиентов (касса)
- контроль запасов топлива
- управлять цистернами и колонками
- организовать пополнение топлива
- вести историю операций и статистику
- обрабатывать аварийные ситуации

Программа работает в виде меню с выбором действий и работает в непрерывном цикле до выхода пользователя.

2. Топливо и цистерны

2.1 Типы топлива

Для заправки использовались следующие виды топлива:

- АИ-92
- АИ-95
- АИ-98
- ДТ (дизельное топливо)

2.2 Цистерны

– Для одного вида топлива может существовать несколько подземных цистерн.

– каждая цистерна имеет:

1. тьма-тьма
2. максимальный объем
3. текущий уровень топлива
4. состояние (включено/отключено)
5. допустимый уровень разработки

Важно:

– Если уровень топлива в цистерне падает ниже минимального порога:

1. цистерна автоматически отключается
2. из нее нельзя сливать топливо

– После пополнения:

1. цистерна не включается автоматически (после пополнения топлива)
2. включение производится вручную через меню

3. Колонки

3.1 Общий вид

– На заправке есть несколько колонок

– Каждая колонка:

1. источники питания нескольких типов топлива
2. Каждый тип топлива (пистолет) подключен к конкретной цистерне

– Не все колонки подключены ко всем цистернам

– 3.2 Схема заправки

– Цистерны:

1. АИ-95 №1 -> колонки 1 - 4
2. АИ-95 №2 -> колонки 5 - 8
3. АИ-92 -> колонки 1 - 6
4. АИ-98 -> колонки 3 - 6
5. ДТ -> колонки 3 - 8

– Всего колонок: 8

– каждая колонка имеет 2–3 пистолета

4. Главное меню программы

В главном меню пользователь может выбрать одно из действий:

- 1) Обслужить клиента (касса)
- 2) Проверить состояние цистерн
- 3) Оформить пополнение топлива
- 4) Баланс и статистика
- 5) История операций
- 6) Перекачка топлива между цистернами
- 7) Включение / отключение цистерн
- 8) Состояние колонок
- 9) EMERGENCY - аварийная ситуация
- 0) Выход

5. Функциональные требования

5.1 Обслуживание клиента (касса)

Система должна:

– принять запрос:

1. вода колонки
2. тьма-тьма
3. количество литров

– проверить:

1. колонки доступности
2. состояние цистерны
3. достаточно топлива

– стоимость покрытия

– запросить подтверждение оплаты

– при подтверждении:

1. списать топливо
2. зафиксировать транзакцию

– при нехватке топлива:

1. сообщить о деньгах

2. не допускать оплату

5.2 Проверка состояния цистерны

Вывод информации:

- тьма-тьма
- четвертые
- максимальный объем
- состояние (включено/отключено)
- отношение на низком уровне

5.3 Заполнение топливом

При прибытии бензовоза:

- выбрать тип топлива
- выбрать конкретную цистерну
- укажите количество литров
- проверьте, не добавляйте максимальный объем
- зафиксировать операцию в анамнезе

5.4 Балансы и статистика

Система должна отображать:

- ожидающий
- количество обслуживаемых автомобилей
- сколько раз покупали каждый тип топлива
- сколько литров каждого топлива продано
- доход при любом виде топлива

5.5 История операций

Вывод последних операций:

- теплые
- цистерна для наполнения
- перекачка топлива
- выключение и включение цистерны

- аварийные события

5.6 Перекачка топлива

- Перекачка возможна только между цистернами одного типа топлива

- Уявся:

1. источник

2. ил

3. фер

- Проверить наличие:

1. достаточно топлива

2. вместимость закрытых цистерн

5.7 Управление цистернами

Пользователь может:

- вручную отключить цистерны

- вручную включают цистерны (если уровень топлива достаточен)

5.8 Состояние колонок

Выводится информация:

- вода колонки

- какие виды топлива доступны

- к каким цистернам они подключены

- какие пистолеты временно не работают (определяется по подключенным цистернам)

5.9 Аварийная ситуация (Чрезвычайная ситуация)

- При подтверждении аварии:

1. все цистерны блокируются

2. заправка прекращает работу

3. фиксируется аварийное событие

4. имитируется вызов экстренных служб

- Выход из аварийного режима возможен только вручную.

- При выходе из аварийного режима цистерны автоматически НЕ разблокируются.

6. Хранение данных

Все данные обязательно указываются в файлах (рекомендуется JSON):

- состояние цистерны
- оксиднок
- баланс и статистика
- история операций

Перезапуск программы не должен сбрасывать состояние заправки.

Содержание

Введение.....	9
1. Основная часть.....	10
1.1 Общая архитектура и структура данных	10
1.2 Модуль управления цистернами (логика хранения).....	12
1.3 Модуль обслуживания клиентов (логика продаж).....	13
1.4 Модуль журналирования и истории операций	15
1.5 Реализация аварийного режима.....	16
1.6 Взаимодействие компонентов системы.....	18
2. Результаты работы программы.....	21
Заключение.....	31
Список используемых источников.....	32

Введение

Целью данной лабораторной работы является проектирование и реализация программного прототипа системы управления автозаправочной станцией (АЗС). Проект направлен на закрепление навыков объектно-ориентированного программирования (ООП), работы с файлами (сериализация данных), обработки пользовательского ввода, а также создания модульной, легко поддерживаемой структуры программы, имитирующей реальные бизнес-процессы.

Автоматизация процессов на АЗС — ключевой фактор повышения эффективности, точности учёта и контроля. Данный прототип моделирует основные операции: отпуск топлива клиентам, управление складскими запасами (подземными цистернами), ведение финансовой статистики, логирование всех операций и обработку аварийных ситуаций. Полученные знания и архитектурные решения применимы в разработке широкого спектра коммерческих и логистических информационных систем.

Система должна выполнять функции кассы, пульта управления резервуарами, журнала событий и панели мониторинга. Все данные сохраняются между запусками программы. Особое внимание уделяется корректной обработке аварийных и критических ситуаций, таких как низкий уровень топлива или ручная блокировка всех систем.

1. Основная часть

1.1 Общая архитектура и структура данных

Программа построена вокруг одного главного класса AZSManager, который инкапсулирует всё состояние и логику работы АЗС. Такой подход соответствует принципам ООП и обеспечивает чёткое разделение ответственности. Класс выступает в роли контроллера, обрабатывающего пользовательский ввод из консольного меню и делегирующего выполнение задач соответствующим методам.

Для обеспечения сохранности данных между сеансами работы программа использует формат JSON и файл azs_data.json. Это позволяет избежать потери состояния (остатки топлива, статистика, история) при перезапуске программы.

```
13 DEFAULT_DATA = {
14     "cisterns": [
15         {"id": 1, "fuel_type": "АИ-95", "max_volume": 20000, "current_volume": 18000, "enabled": True},
16         {"id": 2, "fuel_type": "АИ-95", "max_volume": 20000, "current_volume": 1200, "enabled": False},
17         {"id": 3, "fuel_type": "АИ-92", "max_volume": 20000, "current_volume": 12400, "enabled": True},
18         {"id": 4, "fuel_type": "АИ-98", "max_volume": 15000, "current_volume": 10000, "enabled": False},
19         {"id": 5, "fuel_type": "ДТ", "max_volume": 25000, "current_volume": 15600, "enabled": True}
20     ],
21     "columns": {
22         1: ["АИ-92", "АИ-95"],
23         2: ["АИ-92", "АИ-95"],
24         3: ["АИ-92", "АИ-95", "АИ-98", "ДТ"],
25         4: ["АИ-92", "АИ-95", "АИ-98", "ДТ"],
26         5: ["АИ-92", "АИ-95", "АИ-98", "ДТ"],
27         6: ["АИ-92", "АИ-95", "АИ-98", "ДТ"],
28         7: ["АИ-95", "ДТ"],
29         8: ["АИ-95", "ДТ"]
30     },
31     "cistern_mapping": {
32         "АИ-92": [3],
33         "АИ-95": [1, 2],
34         "АИ-98": [4],
35         "ДТ": [5]
36     },
37     "stats": {
38         "total_cars": 0,
39         "total_income": 0.0,
40         "fuel_sold": {ft: 0 for ft in FUEL_TYPES},
41         "fuel_income": {ft: 0.0 for ft in FUEL_TYPES},
42         "transactions_count": {ft: 0 for ft in FUEL_TYPES}
43     },
44     "history": [],
45     "emergency": False
46 }
```

Рисунок 1 – листинг программы для структуры данных

- cisterns — список словарей, где каждый словарь представляет собой виртуальную цистерну. Поле enabled (True/False) — ключевое для логики работы: если цистерна отключена, отпуск топлива из нее невозможен.

- `columns` — словарь, где ключом является номер колонки (1-8), а значением — список типов топлива, которые можно отпускать с этой колонки. Например, 1: ["АИ-92", "АИ-95"] означает, что с колонки №1 можно продавать АИ-92 и АИ-95.
- `cistern_mapping` — словарь, определяющий, какие конкретные цистерны (по их id) отвечают за хранение определенного типа топлива. Например, "АИ-95": [1, 2] означает, что топливо АИ-95 хранится в цистернах с ID 1 и 2. Это позволяет системе иметь несколько резервуаров для одного вида топлива.
- `stats` — централизованное хранилище всей статистики. Фиксация данных здесь, а не в отдельных переменных, упрощает их сохранение и загрузку.
- `history` — список для последовательного хранения записей о всех важных событиях. Каждая запись — это словарь с временной меткой, типом действия и деталями.
- `emergency` — булева переменная, играющая роль глобального флага состояния системы. Когда она равна `True`, основные функции продажи и управления блокируются.

Принцип инициализации и сохранения:

- При создании экземпляра `AZSManager` вызывается метод `load_data()`.

```

61     def load_data(self):
62         if os.path.exists(DATA_FILE):
63             with open(DATA_FILE, "r", encoding="utf-8") as f:
64                 self.data = json.load(f)
65         else:
66             self.data = DEFAULT_DATA.copy()
67             self.save_data()

```

Рисунок 2 – листинг программы для загрузки данных

1. `os.path.exists(DATA_FILE)` — проверка физического наличия файла на диске. Это необходимо, чтобы отличить первый запуск программы от последующих.
2. Контекстный менеджер `with open(...) as f:` — обеспечивает безопасную работу с файлом. Он гарантирует, что файл будет корректно закрыт после чтения, даже если в процессе произойдет ошибка.
3. `json.load(f)` — функция десериализации. Она "распаковывает" текстовый JSON из файла `f` обратно в сложную структуру данных Python (словари, списки) и присваивает ее атрибуту `self.data`.
4. `DEFAULT_DATA.copy()` — создание глубокой копии (через `copy.deepcopy`, которую делает `.copy()` для словаря верхнего уровня с простыми вложенными

структурами). Это критически важно, чтобы изменения в self.data в ходе работы программы не затрагивали исходную константу DEFAULT_DATA.

- Любое изменение состояния (продажа, пополнение, перекачка) завершается вызовом save_data(), что гарантирует актуальность сохранённой информации.

```
69     def save_data(self):
70         with open(DATA_FILE, "w", encoding="utf-8") as f:
71             json.dump(self.data, f, ensure_ascii=False, indent=2)
--
```

Рисунок 3 – листинг программы для сохранения данных

- "w" — режим записи. Если файл существует, он будет полностью перезаписан.
- json.dump(obj, file, ...) — функция сериализации. Она преобразует (dump — "сбрасывает") объект Python self.data в текстовый JSON-формат и сразу записывает его в открытый файловый объект f.
- ensure_ascii=False — позволяет сохранять символы кириллицы (например, в истории операций) как есть, а не в виде Unicode escape-последовательностей (\u0430).
- indent=2 — форматирует выходной JSON с отступами в 2 пробела, что делает файл azs_data.json легко читаемым для человека при отладке.

1.2 Модуль управления цистернами (логика хранения)

Цистерна моделируется словарём с полями: id, fuel_type, max_volume, current_volume, enabled. Основная бизнес-логика управления их состоянием сосредоточена в двух методах:

1. Автоматический контроль уровня (Автоотключение):
Метод check_cisterns_levels() вычисляет минимально допустимый уровень (10% от ёмкости) и при его достижении отключает цистерну, предотвращая работу всухую и фиксируя событие в истории.

```
83     def check_cisterns_levels(self):
84         for cistern in self.data["cisterns"]:
85             min_level = cistern["max_volume"] * MIN_LEVEL_PERCENT / 100
86             if cistern["current_volume"] < min_level and cistern["enabled"]:
87                 cistern["enabled"] = False
88                 self.add_history("Автоотключение цистерны",
89                                f"{cistern['fuel_type']} №{cistern['id']} - низкий
                                уровень")
```

Рисунок 4 – листинг программы автоматического контроля уровня топлива

2. Ручное управление цистернами:

Метод `manage_cisterns()` предоставляет интерфейс для оператора. При попытке включения цистерны с уровнем ниже порога система выдаёт предупреждение, но позволяет оператору принять решение.

```
83     def check_cisterns_levels(self):
84         for cistern in self.data["cisterns"]:
85             min_level = cistern["max_volume"] * MIN_LEVEL_PERCENT / 100
86             if cistern["current_volume"] < min_level and cistern["enabled"]:
87                 cistern["enabled"] = False
88                 self.add_history("Автоотключение цистерны",
89                                 f"{cistern['fuel_type']} №{cistern['id']} – низкий
                                     уровень")
```

Рисунок 5 – листинг программы ручного включения/отключения цистерн

Вспомогательные методы для работы с цистернами:

Для удобства работы реализованы методы поиска и фильтрации.

```
98     def get_available_cisterns(self, fuel_type):
99         return [c for c in self.data["cisterns"] if c["fuel_type"] == fuel_type]
100
101     def get_cistern_for_column(self, column_num, fuel_type):
102         if column_num not in self.data["columns"] or fuel_type not in
            self.data["columns"][column_num]:
103             return None
104
105         cistern_ids = self.data["cistern_mapping"][fuel_type]
106         for cistern in self.data["cisterns"]:
107             if cistern["id"] in cistern_ids and cistern["enabled"]:
108                 return cistern
109         return None
```

Рисунок 6 – листинг программы поиска и фильтрации

1.3 Модуль обслуживания клиентов (логика продаж)

Метод `serve_customer()` реализует полный цикл продажи, включая многоуровневую валидацию.

```

136     def serve_customer(self): #обслужить клиента
137         if self.data["emergency"]:
138             print("Работа невозможна в аварийном режиме!")
139             return
140
141         print("\n--- Обслуживание клиента ---")
142
143         # Выбор колонки
144         print("\nДоступные колонки:")
145         for i in range(1, 9):
146             print(f"{i}) Колонка {i}")
147
148         try:
149             column_num = int(input("\nВыберите колонки: "))
150             if column_num not in range(1, 9):
151                 print("Ошибка: неверный номер колонки")
152                 return
153         except ValueError:
154             print("Ошибка: введите число")
155             return
156
157         available_fuels = self.data["columns"][column_num]
158         print(f"\nКолонка {column_num}")
159         print("Доступные виды топлива:")
160
161         for i, fuel in enumerate(available_fuels, 1):
162             cistern = self.get_cistern_for_column(column_num, fuel)
163             status = "✓" if cistern and cistern["enabled"] else "× (цистерна отключена)"
164             print(f"{i}) {fuel} {status}")
165
166         try:
167             choice = int(input("\nВыберите тип топлива: "))
168             if choice not in range(1, len(available_fuels) + 1):
169                 print("Ошибка: неверный выбор")
170                 return
171             fuel_type = available_fuels[choice - 1]
172         except ValueError:
173             print("Ошибка: введите число")
174             return

```

Рисунок 7 – листинг программы для обслуживания клиента (1 часть)

1. try: Начинаем блок "попытки" выполнить код, который может вызвать исключение.
2. float(input(...)): Пытаемся преобразовать строку ввода в число с плавающей точкой.
3. Возможные сценарии:
 - Пользователь ввел "40" → float("40") успешно вернет 40.0
 - Пользователь ввел "сорок" → float("сорок") вызовет ValueError
4. except ValueError: Ловим конкретное исключение "неверное значение".
5. Логические проверки: Даже если преобразование прошло успешно, проверяем бизнес-правила (положительность, достаточность).


```

176     cistern = self.get_cistern_for_column(column_num, fuel_type)
177     if not cistern:
178         print(f"\nОШИБКА: Нет доступной цистерны для {fuel_type}")
179         return
180     if not cistern["enabled"]:
181         print(f"\nОШИБКА: Цистерна {cistern['fuel_type']} №{cistern['id']} отключена.")
182         print("Отпуск топлива невозможен.")
183         return
184
185     try:
186         liters = float(input("\nВведите количество литров: "))
187         if liters <= 0:
188             print("Ошибка: количество должно быть положительным")
189             return
190         if liters > cistern["current_volume"]:
191             print(f"Ошибка: недостаточно топлива в цистерне (доступно: {cistern['current_volume']} л)")
192             return
193     except ValueError:
194         print("Ошибка: введите число")
195         return
196
197     price = PRICES[fuel_type]
198     cost = liters * price
199
200     print(f"\nСтоимость:")
201     print(f"{liters} л × {price:.2f} ₺ = {cost:.2f} ₺")
202
203     confirm = input("\nПодтвердить оплату? (y/n): ").lower()
204     if confirm != 'y':
205         print("Операция отменена")
206         return
207
208     cistern["current_volume"] -= liters
209
210     self.data["stats"]["total_cars"] += 1
211     self.data["stats"]["total_income"] += cost
212     self.data["stats"]["fuel_sold"][fuel_type] += liters
213     self.data["stats"]["fuel_income"][fuel_type] += cost
214     self.data["stats"]["transactions_count"][fuel_type] += 1
215
216     self.add_history("Продажа топлива",
217                    f"Колонка {column_num}, {fuel_type}, {liters} л, {cost:.2f} ₺")
218
219     self.save_data()
220     print("\nОперация выполнена успешно.")

```

Рисунок 8 – листинг программы для обслуживания клиента (2 часть)

1.4 Модуль журналирования и истории операций

Система ведёт подробный журнал всех значимых событий.

Метод `add_history()` обеспечивает единообразное логирование.

```

73     def add_history(self, action, details):
74         timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
75         self.data["history"].append({
76             "timestamp": timestamp,
77             "action": action,
78             "details": details
79         })
80         if len(self.data["history"]) > 100:
81             self.data["history"] = self.data["history"][-100:]

```

Рисунок 9 – листинг программы для истории событий

История позволяет отследить последовательность всех операций, что критически важно для:

- Аудита и отладки (поиска причин ошибок).
- Восстановления данных (понимания, какие действия привели к текущему состоянию).
- Формирования отчётов (анализ активности за период).

1.5 Реализация аварийного режима

Аварийный режим — критически важная функция безопасности. Его реализация демонстрирует принцип разделения ответственности и изменения поведения системы на основе состояния.

```
505     def emergency_procedure(self): #авария
506         print("\n--- АВАРИЙНАЯ СИТУАЦИЯ ---")
507         print("\n! ВНИМАНИЕ! АВАРИЙНАЯ СИТУАЦИЯ! !")
508         print("1) Все цистерны будут заблокированы")
509         print("2) Заправка прекращает работу")
510         print("3) Будет зафиксировано аварийное событие")
511         print("4) Имитируется вызов аварийных служб")
512
513         confirm = input("\nВы уверены? Это серьезное действие! (y/n): ").lower()
514         if confirm != 'y':
515             print("Отмена аварийной процедуры")
516             return
517
518         self.data["emergency"] = True
519
520         for cistern in self.data["cisterns"]:
521             cistern["enabled"] = False
522
523         self.add_history("АВАРИЙНЫЙ РЕЖИМ", "Активирован аварийный режим, все
524             цистерны отключены")
525
526         self.save_data()
527         print("\n" + "=" * 50)
528         print("АВАРИЙНЫЙ РЕЖИМ АКТИВИРОВАН!")
529         print("=" * 50)
530         print("\nИмитация вызова аварийных служб...")
531         print("01, 02, 03 – вызваны на АЗС <<СеверНефть>>")
532         print("\nОжидание прибытия спасателей...")
```

Рисунок 10 – листинг программы для аварийного режима

Адаптация главного меню под аварийный режим:

В методе `run()` главный цикл меняет своё поведение в зависимости от флага `emergency`. Это пример простейшего конечного автомата (state machine).


```

553     def run(self): #запуск программы и меню
554         while True:
555             self.clear_screen()
556             self.print_header()
557
558             if self.data["emergency"]:
559                 print("\n" + "-" * 50)
560                 print("В аварийном режиме доступны только:")
561                 print("2) Проверить состояние цистерн")
562                 print("5) История операций")
563                 print("10) Выход из аварийного режима")
564                 print("0) Выход")
565             else:
566                 print("\n" + "-" * 50)
567                 print("Выберите действия:")
568                 print("1) Обслужить клиента (касса)")
569                 print("2) Проверить состояние цистерн")
570                 print("3) Оформить пополнение топлива")
571                 print("4) Баланс и статистика")
572                 print("5) История операций")
573                 print("6) Перекачка топлива между цистернами")
574                 print("7) Включение / отключение цистерн")
575                 print("8) Состояние колонок")
576                 print("9) EMERGENCY – аварийная ситуация")
577                 print("0) Выход")
578
579             print("-" * 50)

```

Рисунок 11 – листинг программы для меню при аварийном режиме

Выход из аварийного режима:

Важно, что выход из аварийного режима не приводит к автоматическому включению цистерн, требуя сознательных действий оператора для восстановления работы каждой цистерны вручную — это соответствует принципам безопасности ("двухэтапное восстановление").

```

533     def exit_emergency(self): #выход из аварийного режима
534         if not self.data["emergency"]:
535             print("Аварийный режим не активен")
536             return
537
538         print("\n--- Выход из аварийного режима ---")
539         print("ВНИМАНИЕ: После выхода из аварийного режима")
540         print("цистерны НЕ будут автоматически включены!")
541         print("Их нужно будет включить вручную через меню.")
542
543         confirm = input("\nВыйти из аварийного режима? (y/n): ").lower()
544         if confirm != 'y':
545             print("Отмена")
546             return
547
548         self.data["emergency"] = False
549         self.add_history("Выход из аварийного режима", "Аварийный режим
                    деактивирован")
550         self.save_data()
551         print("\nАварийный режим деактивирован.")

```

Рисунок 12 – листинг программы для выхода из аварийного режима

1.6 Взаимодействие компонентов системы

На схеме ниже показано, как основные компоненты системы взаимодействуют при обработке типичного запроса на продажу топлива. Схема представляет собой визуализацию полного жизненного цикла обработки запроса на продажу топлива в системе управления АЗС. Это последовательность проверок и действий, которые система выполняет от момента выбора колонки клиентом до фиксации успешной операции.

1.1. Инициирование процесса (блоки А-В-С)

- А → В: Клиент через консольный интерфейс выбирает колонку и тип топлива. Этот ввод поступает в главное меню класса AZSManager.
- В → С: Главное меню передает управление методу serve_customer() - основному обработчику логики продаж.

1.2. Каскадная валидация (блоки D-K)

Это ядро системы безопасности и целостности данных. Каждый ромб (ромбовидный блок) представляет проверку:

1. Проверка D: "Доступны ли колонка и тип топлива?"
 - Что проверяется: Соответствует ли введенный номер колонки существующим (1-8) и поддерживает ли эта колонка выбранный тип топлива согласно схеме columns.
 - Пример: Если клиент выбрал колонку 5 и топливо АИ-98, система проверяет, есть ли "АИ-98" в списке columns[5].
2. Проверка F: "Есть ли активная цистерна?"

- Что проверяется: Вызов метода `get_cistern_for_column()`, который:
 - а) Определяет по `cistern_mapping`, какие цистерны хранят данный тип топлива
 - б) Проверяет среди этих цистерн первую с флагом `enabled=True`
- Важность: Даже если топливо есть на колонке, оно не может быть отпущено из отключенной цистерны.

3. Проверка Н: "Достаточно ли топлива?"

- Что проверяется: Сравнение запрашиваемого объема с `current_volume` найденной активной цистерны.
- Бизнес-правило: Нельзя продать больше, чем есть в наличии, даже если цистерна активна.

4. Проверка К: "Подтверждение оплаты?"

- Что проверяется: Финальное согласие клиента/оператора после показа стоимости.
- Важность: Последний "рубеж безопасности" перед необратимым изменением данных.

1.3. Исполнение операции (блоки М-Q)

Если все проверки пройдены и получено подтверждение, система выполняет атомарную группу изменений:

- N (Списание): `cistern["current_volume"] -= liters`
- (Статистика): Обновление `stats` (доходы, счетчики)
- P (История): Добавление записи в `history`

Ключевой момент: Все три изменения происходят в памяти как единое логическое действие. Только после этого:

- Q (Сохранение): Вызов `save_data()` сериализует ВСЕ изменения в файл `azs_data.json`. Это обеспечивает целостность данных - либо сохраняются все изменения сразу, либо ни одно (при ошибке записи).
- 1.4. Завершение процесса (блоки R, S, E, G, I, L)
- R → S: Успешное завершение возвращает в главное меню.
- E, G, I, L → S: Любая ошибка на этапе валидации или отмена также возвращает в меню, но без изменения состояния системы.

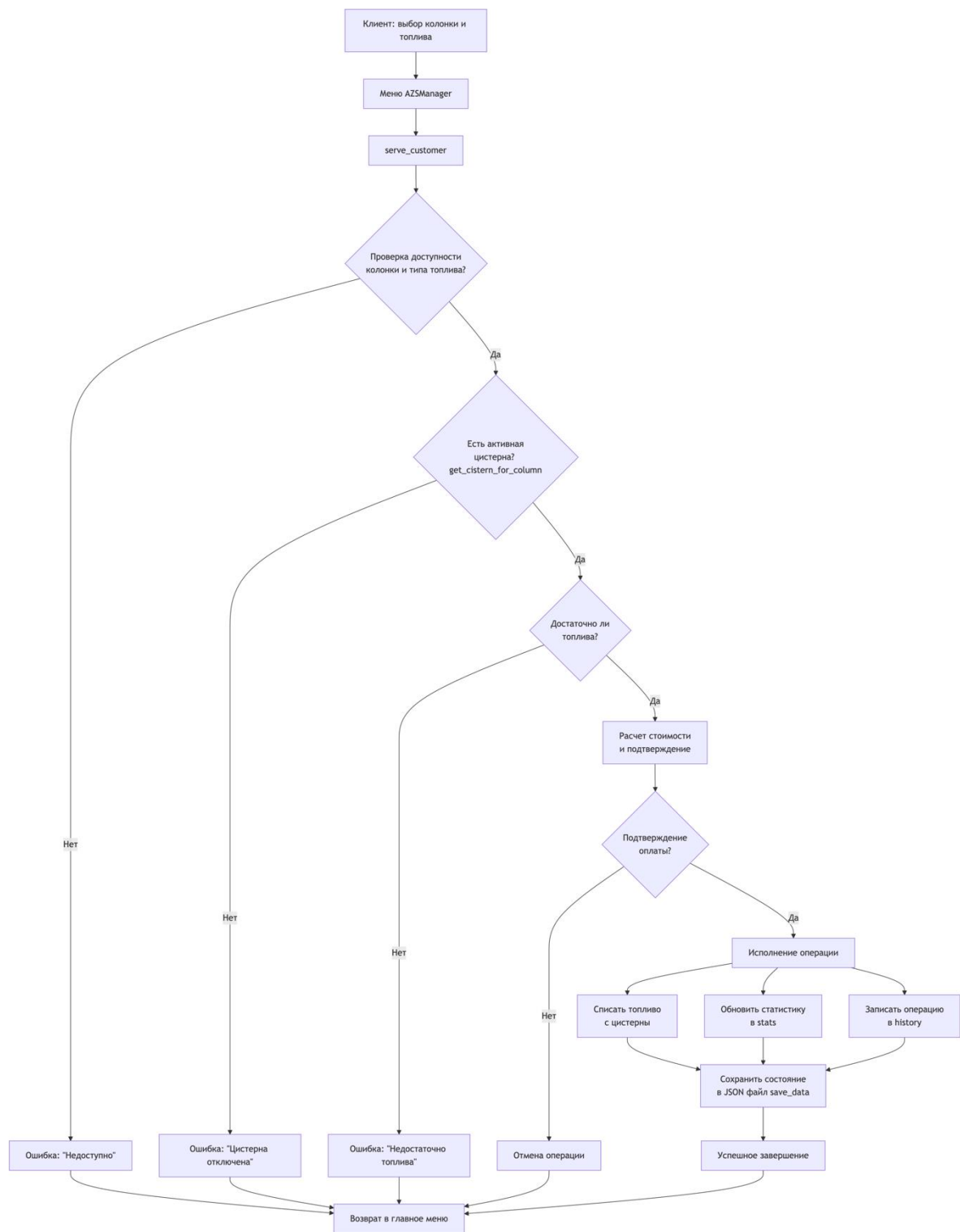
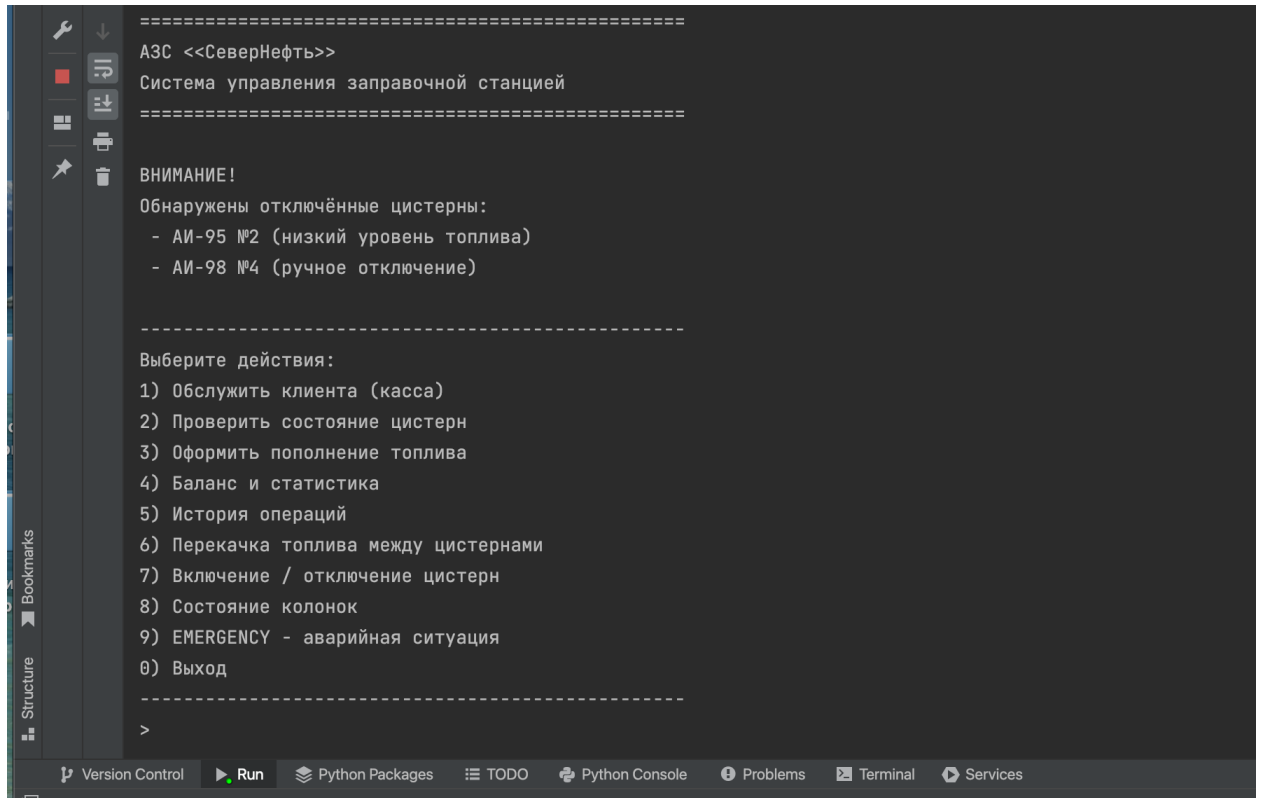


Рисунок 13 – схема компонентов системы при продаже топлива

2. Результаты работы программы

Блок 0: реализация главного меню и интерфейса.

Результат реализации: Создана интуитивно понятная консольная система управления с разделением на нормальный и аварийный режимы работы.



```
=====
АЗС <<СеверНефть>>
Система управления заправочной станцией
=====

ВНИМАНИЕ!
Обнаружены отключённые цистерны:
- АИ-95 №2 (низкий уровень топлива)
- АИ-98 №4 (ручное отключение)

-----

Выберите действия:
1) Обслужить клиента (касса)
2) Проверить состояние цистерн
3) Оформить пополнение топлива
4) Баланс и статистика
5) История операций
6) Перекачка топлива между цистернами
7) Включение / отключение цистерн
8) Состояние колонок
9) EMERGENCY - аварийная ситуация
0) Выход

-----
>
```

Рисунок 14 – основное меню

Ключевые особенности:

- Автоматическое определение и отображение проблемных цистерн при запуске
- Раздельное меню для нормального и аварийного режимов
- Единообразное оформление с разделителями для улучшения читаемости
- Обработка некорректного ввода с возвратом в меню

Блок 1: Обслуживание клиента (касса).

Результат реализации: Полностью автоматизированный процесс продажи топлива с многоуровневой проверкой.

```
--- Обслуживание клиента ---

Доступные колонки:
1) Колонка 1
2) Колонка 2
3) Колонка 3
4) Колонка 4
5) Колонка 5
6) Колонка 6
7) Колонка 7
8) Колонка 8

Выберите колонки: 1

Колонка 1
Доступные виды топлива:
1) АИ-92 ✓
2) АИ-95 ✓

Выберите тип топлива: 2

Введите количество литров: 20

Стоимость:
20.0 л × 69.30 ₺ = 1386.00 ₺

Подтвердить оплату? (y/n): y

Операция выполнена успешно.
Спасибо за покупку!

Нажмите Enter для возврата в меню...
```

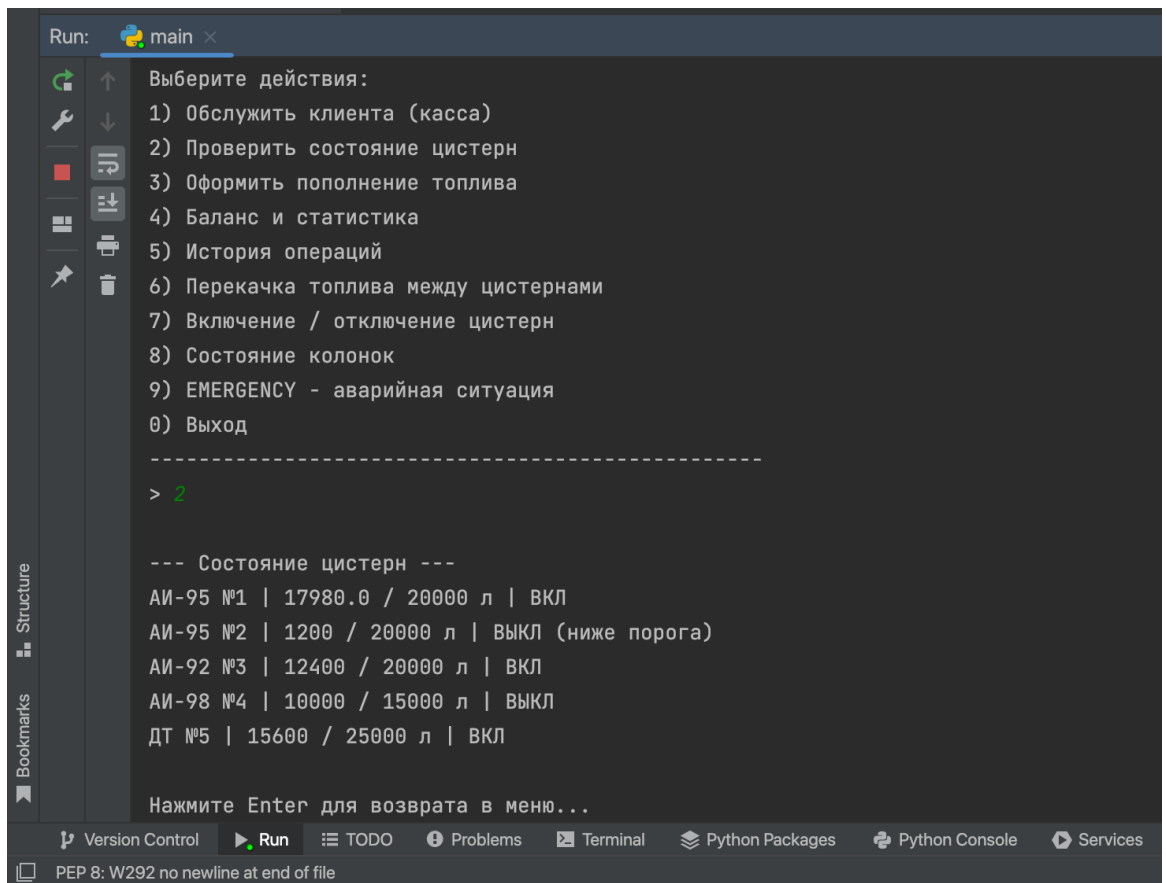
Рисунок 15 – обслуживание клиента

Ключевые особенности:

- Каскадная валидация: колонка → топливо → цистерна → объем → подтверждение
- Автоматический расчет стоимости с форматированием
- Проверка доступности и состояния цистерн в реальном времени
- Индикация статуса цистерны (✓/X) при выборе топлива

Блок 2: Проверка состояния цистерн.

Результат реализации: Комплексная система мониторинга всех цистерн с цветовой индикацией проблем.



```
Run: main x
Выберите действия:
1) Обслужить клиента (касса)
2) Проверить состояние цистерн
3) Оформить пополнение топлива
4) Баланс и статистика
5) История операций
6) Перекачка топлива между цистернами
7) Включение / отключение цистерн
8) Состояние колонок
9) EMERGENCY - аварийная ситуация
0) Выход

-----
> 2

--- Состояние цистерн ---
АИ-95 №1 | 17980.0 / 20000 л | ВКЛ
АИ-95 №2 | 1200 / 20000 л | ВЫКЛ (ниже порога)
АИ-92 №3 | 12400 / 20000 л | ВКЛ
АИ-98 №4 | 10000 / 15000 л | ВЫКЛ
ДТ №5 | 15600 / 25000 л | ВКЛ

Нажмите Enter для возврата в меню...
```

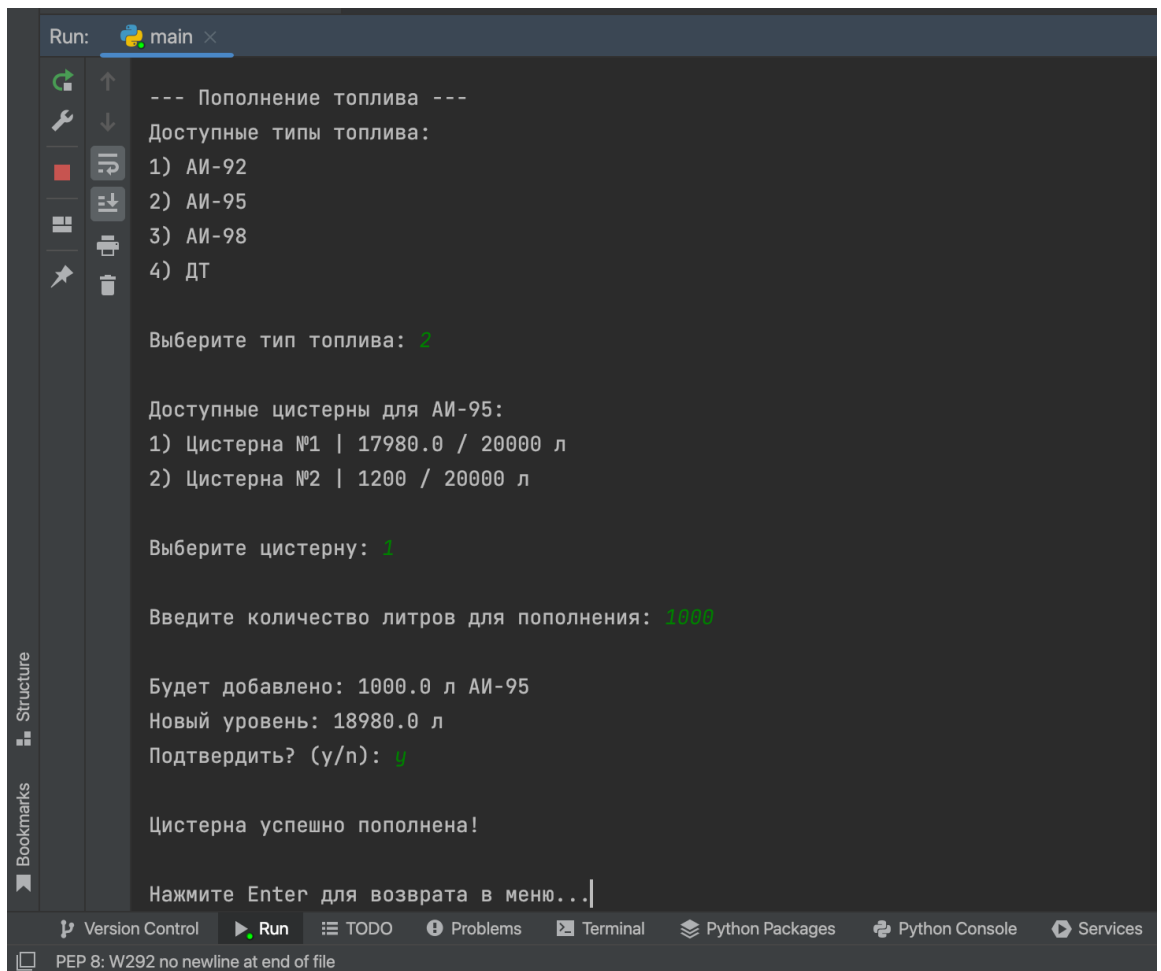
Рисунок 16 – состояние цистерн

Ключевые особенности:

- Отображение текущего и максимального объема для каждой цистерны
- Четкая индикация состояния (ВКЛ/ВЫКЛ)
- Автоматическое определение причины отключения ("ниже порога")
- Группировка цистерн по типам топлива
- Вычисление процента заполнения (неявно, через соотношение объемов)

Блок 3: Оформление пополнения топлива.

Результат реализации: Безопасная система приема топлива от бензовоза с проверкой ограничений. Для безопасности работы АЗС система запрашивает подтверждение операции у сотрудника.



```
Run: main x
--- Пополнение топлива ---
Доступные типы топлива:
1) АИ-92
2) АИ-95
3) АИ-98
4) ДТ

Выберите тип топлива: 2

Доступные цистерны для АИ-95:
1) Цистерна №1 | 17980.0 / 20000 л
2) Цистерна №2 | 1200 / 20000 л

Выберите цистерну: 1

Введите количество литров для пополнения: 1000

Будет добавлено: 1000.0 л АИ-95
Новый уровень: 18980.0 л
Подтвердить? (y/n): y

Цистерна успешно пополнена!

Нажмите Enter для возврата в меню...|
```

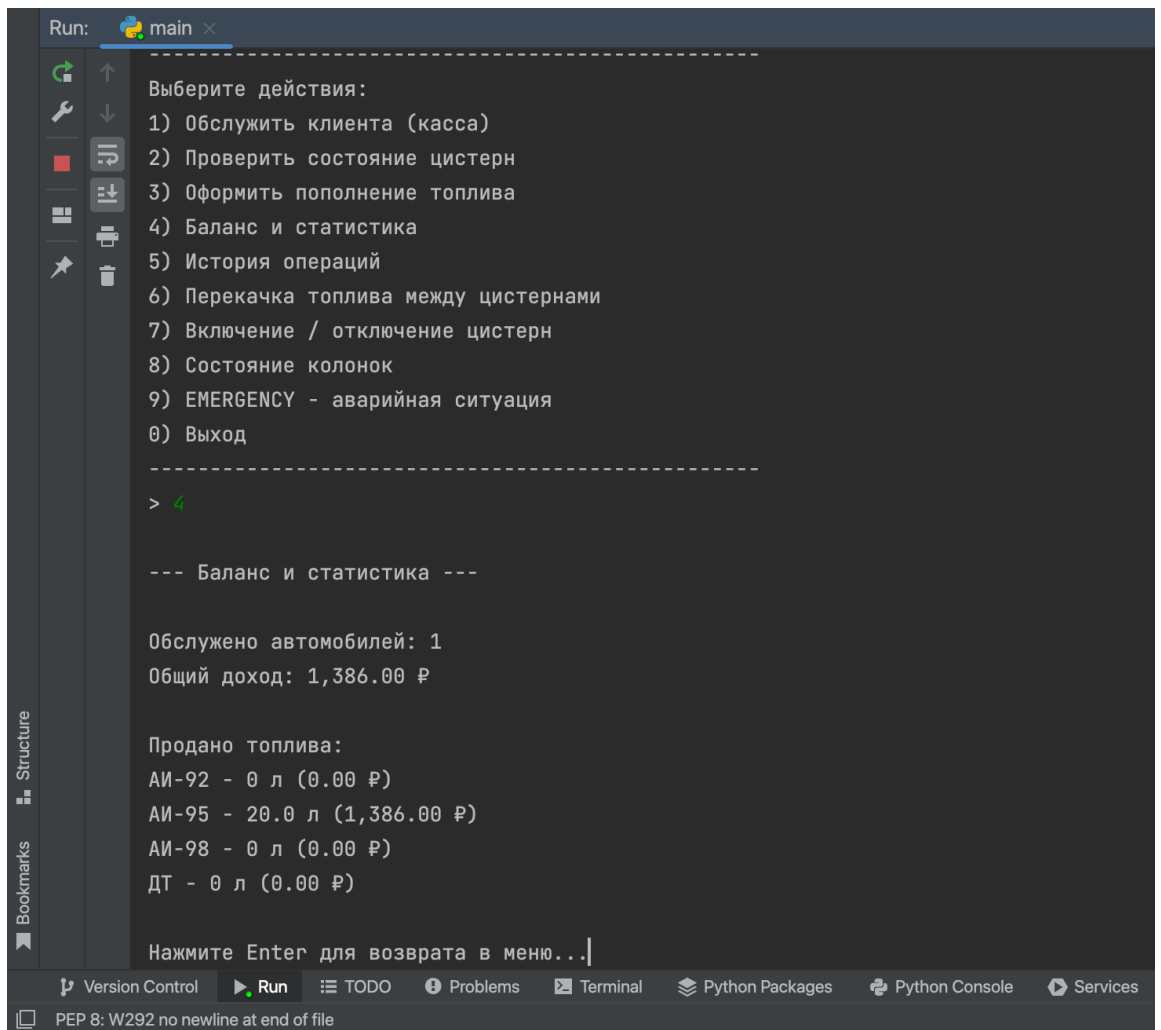
Рисунок 17 – пополнение цистерн

Ключевые особенности:

- Защита от переполнения цистерн
- Подтверждение операции перед исполнением
- Цистерна остается отключенной после пополнения (требование ТЗ)
- Расчет доступного объема для добавления
- Фиксация операции в истории

Блок 4: Баланс и статистика.

Результат реализации: Детальная финансовая и операционная отчетность в реальном времени.



```
Run: main x
-----
Выберите действия:
1) Обслужить клиента (касса)
2) Проверить состояние цистерн
3) Оформить пополнение топлива
4) Баланс и статистика
5) История операций
6) Перекачка топлива между цистернами
7) Включение / отключение цистерн
8) Состояние колонок
9) EMERGENCY - аварийная ситуация
0) Выход
-----
> 

--- Баланс и статистика ---

Обслужено автомобилей: 1
Общий доход: 1,386.00 ₺

Продано топлива:
AI-92 - 0 л (0.00 ₺)
AI-95 - 20.0 л (1,386.00 ₺)
AI-98 - 0 л (0.00 ₺)
ДТ - 0 л (0.00 ₺)

Нажмите Enter для возврата в меню...|
Version Control Run TODO Problems Terminal Python Packages Python Console Services
PEP 8: W292 no newline at end of file
```

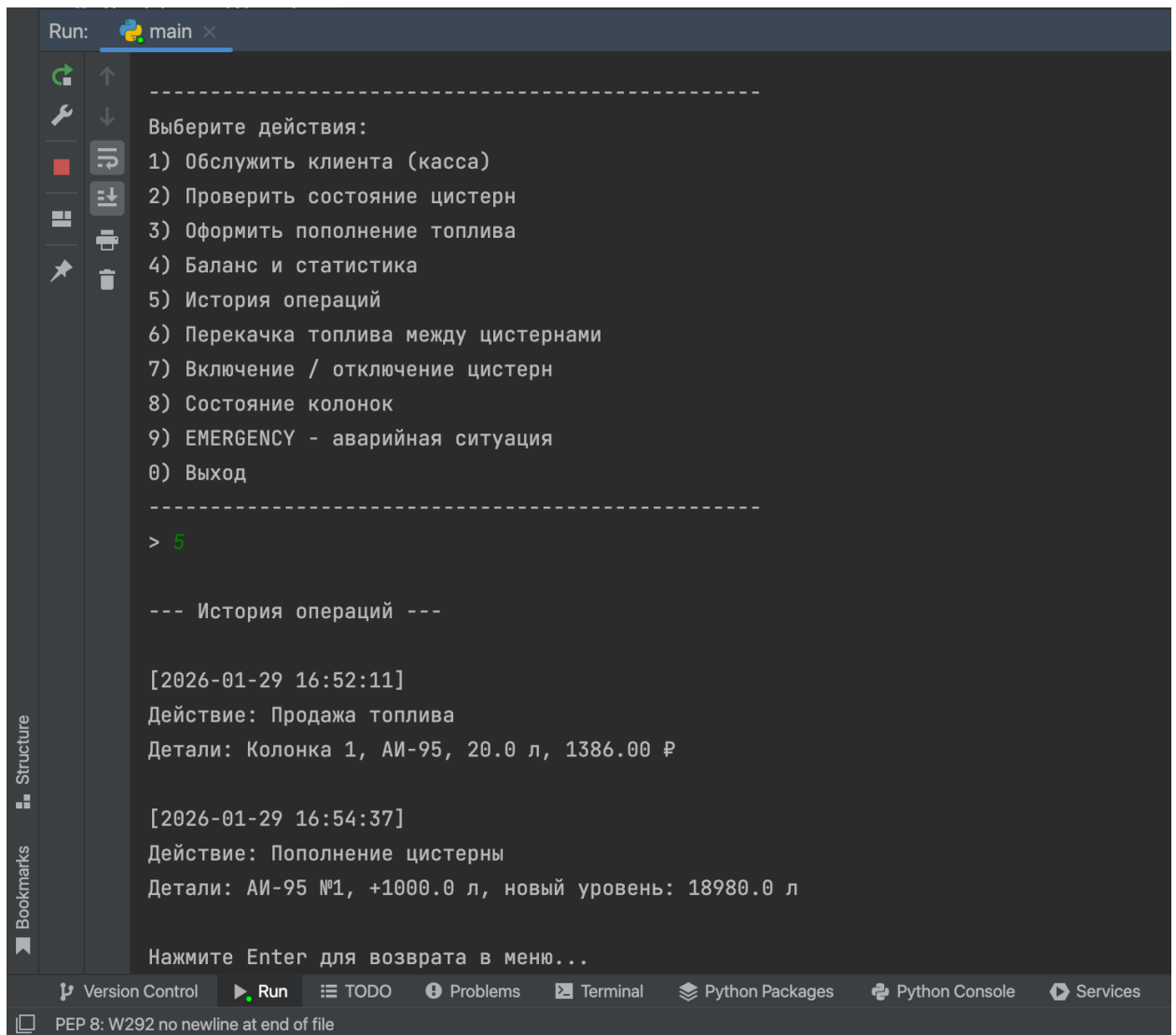
Рисунок 18 – баланс и статистика

Ключевые особенности:

- Форматирование чисел с разделителями тысяч
- Детализация по каждому типу топлива (литры + доход)
- Расчет производных метрик (средний чек)
- Накопительный учет с сохранением между сеансами
- Подготовка данных для бухгалтерской отчетности

Блок 5: История операций.

Результат реализации: Полнофункциональный журнал аудита всех действий в системе.



```
Run: main x
-----
Выберите действия:
1) Обслужить клиента (касса)
2) Проверить состояние цистерн
3) Оформить пополнение топлива
4) Баланс и статистика
5) История операций
6) Перекачка топлива между цистернами
7) Включение / отключение цистерн
8) Состояние колонок
9) EMERGENCY - аварийная ситуация
0) Выход
-----
> 5

--- История операций ---

[2026-01-29 16:52:11]
Действие: Продажа топлива
Детали: Колонка 1, АИ-95, 20.0 л, 1386.00 ₽

[2026-01-29 16:54:37]
Действие: Пополнение цистерны
Детали: АИ-95 №1, +1000.0 л, новый уровень: 18980.0 л

Нажмите Enter для возврата в меню...
```

Рисунок 19 – история операций

Ключевые особенности:

- Хронологический порядок записей с timestamp
- Классификация операций по типам
- Детализированное описание каждого события
- Автоматическое ограничение истории (последние 100 записей)
- Поддержка кириллицы в записях

Блок 6: Перекачка топлива между цистернами

Результат реализации: Безопасный механизм перераспределения топлива между резервуарами.

```
Run: main x
--- Перекачка топлива ---
Выберите тип топлива для перекачки:
1) АИ-92
2) АИ-95
3) АИ-98
4) ДТ

Выберите тип топлива: 2

Доступные цистерны АИ-95:
1) Цистерна №1 | 18980.0 / 20000 л | ВКЛ
2) Цистерна №2 | 1200 / 20000 л | ВЫКЛ

Выберите исходную цистерну (откуда перекачивать): 1
Выберите целевую цистерну (куда перекачивать): 2

Введите количество литров для перекачки: 100

Перекачка 100.0 л АИ-95:
Из: Цистерна №1 (18980.0 л)
В: Цистерна №2 (1200 л)
Подтвердить? (y/n): y

Перекачка выполнена успешно!

Нажмите Enter для возврата в меню...
```

Рисунок 20 – перекачка топлива

Ключевые особенности:

- Проверка типа топлива (перекачка только между однотипными цистернами)
- Контроль достаточности объема в источнике
- Защита от переполнения приемника
- Запрет перекачки в ту же цистерну
- Фиксация операции в истории

Блок 7: Включение и отключение цистерн.

Результат реализации: Система ручного управления состоянием цистерн с контролем безопасности.

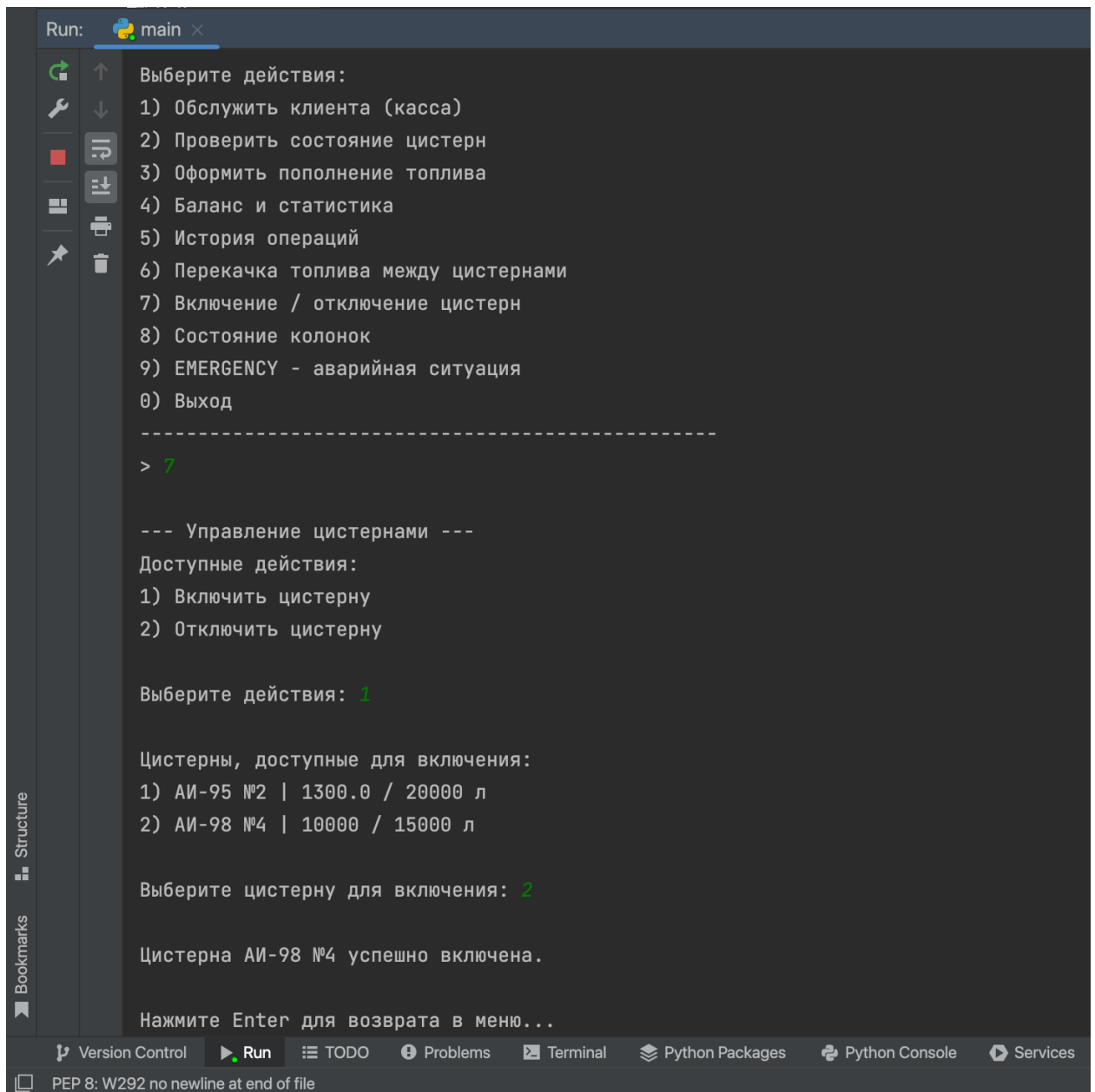


Рисунок 21 – включение цистерны

Ключевые особенности:

- Раздельный интерфейс для включения и отключения
- Фильтрация списка по текущему состоянию цистерн
- Предупреждение при включении цистерны с низким уровнем
- Возможность принудительного включения (ответственность оператора)
- Мгновенное применение изменений

Блок 8: Состояние колонок.

Результат реализации: Визуализация схемы подключения и текущей доступности колонок.

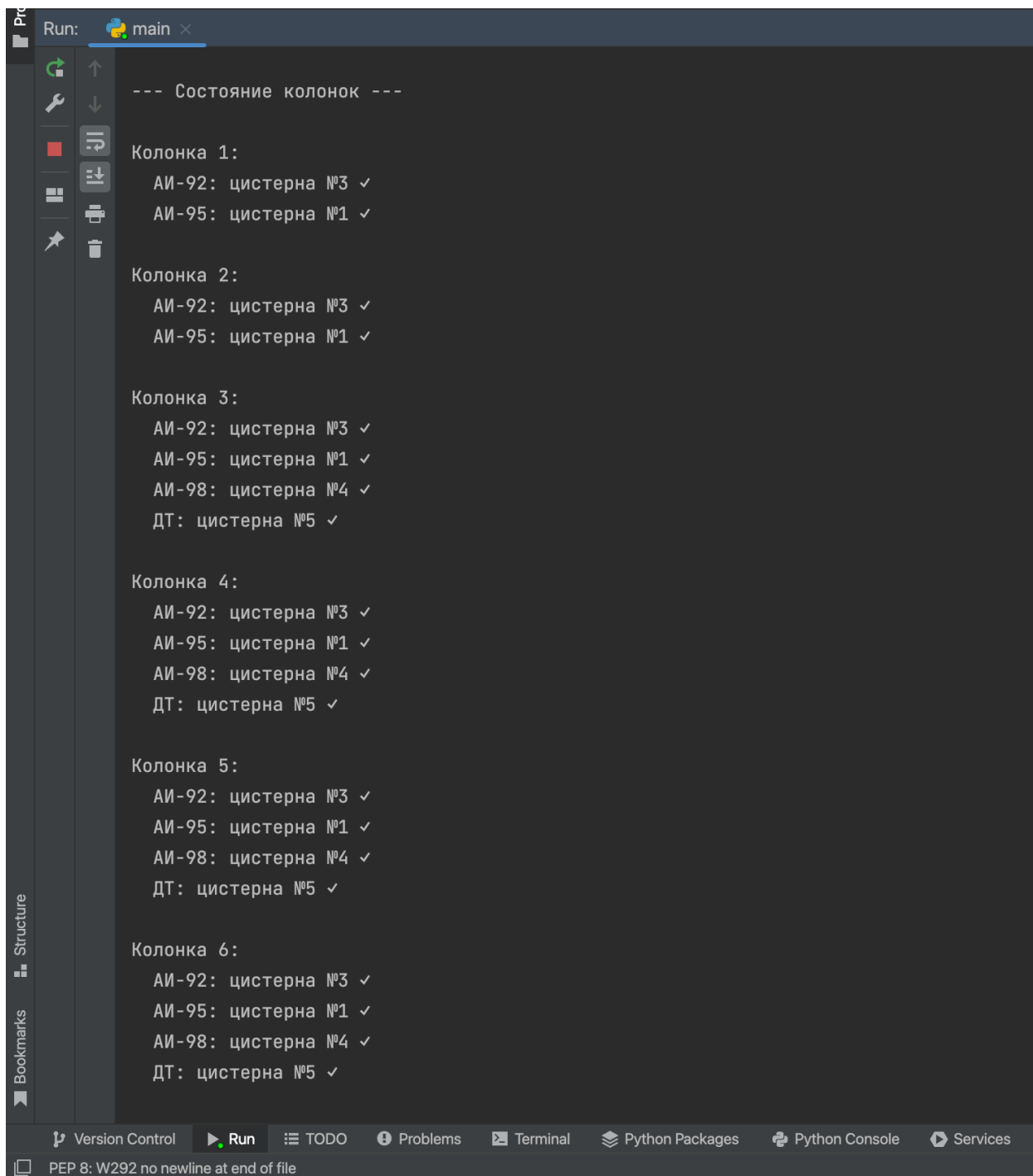


Рисунок 22 – состояние колонок

Ключевые особенности:

- Отображение полной схемы подключения (8 колонок)
- Указание конкретной цистерны для каждого типа топлива
- Индикация доступности (✓/X) с указанием причины
- Группировка по колонкам для удобства восприятия
- Отражение реального состояния системы в данный момент

Блок 9: Аварийный режим.

Результат реализации: Полнофункциональная система аварийного отключения с изолированным режимом работы.

The screenshot shows the PyCharm IDE interface. At the top, there's a tab labeled "main". Below it, a vertical toolbar contains icons for running, debugging, and other actions. The main area is a terminal window displaying Russian text. The text starts with a warning icon and "ВНИМАНИЕ! АВАРИЙНАЯ СИТУАЦИЯ!", followed by a numbered list of four items. Then it asks "Вы уверены? Это серьезное действие! (y/n):" where 'y' has been entered. This is followed by a separator line, "АВАРИЙНЫЙ РЕЖИМ АКТИВИРОВАН!", another separator line, "Имитация вызова аварийных служб...", "01, 02, 03 - вызваны на АЗС <<СеверНефть>>", "Ожидание прибытия спасателей...", "Нажмите Enter для возврата в меню...", another separator line, "АЗС <<СеверНефть>>", "Система управления заправочной станцией", another separator line, another warning icon and "ВНИМАНИЕ! АВАРИЙНЫЙ РЕЖИМ!", "Все цистерны заблокированы. Работа приостановлена.", a dashed line, "В аварийном режиме доступны только:", a list of options (2) Проверить состояние цистерн, 5) История операций, 10) Выход из аварийного режима, 0) Выход, another dashed line, and finally a prompt character ">". At the bottom, the status bar shows "Version Control", "Run", "TODO", "Problems", "Terminal", "Python Packages", "Python Console", and "Services". A small icon on the left indicates a PEP 8 error: "PEP 8: W292 no newline at end of file".

Рисунок 23 – аварийный режим

Ключевые особенности:

- Многоступенчатое подтверждение активации
- Автоматическое отключение ВСЕХ цистерн
- Имитация вызова экстренных служб
- Специальное меню с ограниченным функционалом

Заключение

Достижение цели: В ходе лабораторной работы был успешно разработан полнофункциональный консольный прототип системы управления АЗС. Программа реализует все требования технического задания: от обслуживания клиентов и управления запасами до ведения статистики и обработки аварийных ситуаций.

Архитектурные решения: Применение объектно-ориентированного подхода в виде центрального класса `AZSManager` позволило создать хорошо структурированное и масштабируемое приложение. Использование JSON для хранения состояния обеспечило простоту и надёжность сохранности данных между сеансами работы. Чёткое разделение логики на модули (управление цистернами, продажи, журналирование) упрощает поддержку и развитие системы.

Реализация бизнес-логики: Ключевые процессы АЗС смоделированы с учётом реальных ограничений и требований безопасности:

Каскадная валидация в процессе продажи гарантирует целостность данных.

Автоматический контроль уровня топлива предотвращает ошибки оператора.

Аварийный режим реализован как глобальное состояние системы, кардинально меняющее её поведение, что соответствует практике промышленных систем управления.

Практическая значимость: Работа продемонстрировала ключевые принципы разработки бизнес-приложений: моделирование предметной области, валидацию ввода, разделение данных и логики, устойчивость к ошибкам и ведение журнала операций. Полученный прототип может служить основой для более сложных систем автоматизации торговых и логистических объектов.

Список используемых источников

1. Python Software Foundation. Документация по модулю json [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/json.html>
2. Python Software Foundation. Документация по модулю datetime [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/datetime.html>
3. Python Software Foundation. Документация по модулю os [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/library/os.html>
4. Словари (dict) в Python. Программирование на Python [Электронный ресурс]. – Режим доступа: <https://pythonworld.ru/tipy-dannyx-v-python/slovari-dict-funkcii-i-metody-slovaroj.html>
5. Работа с файлами в Python. Введение [Электронный ресурс]. – Режим доступа: <https://pythonist.ru/rabota-s-fajlami-v-python-vvedenie/>